

Segurança em Computação

INE5429-07208

Thaís Bardini Idalino

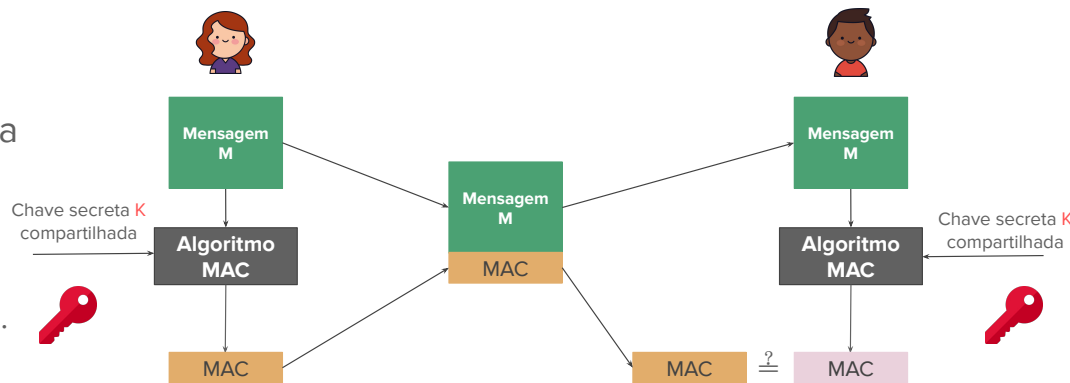
Assinaturas digitais e protocolos criptográficos

Sumário

- Assinaturas digitais
 - Princípios básicos
 - Assinatura digital com RSA
 - Assinatura digital com ECDSA
- Protocolos criptográficos
- Atividades práticas

Message Authentication Code (MAC)

- MACs protegem Alice e Bob contra terceiros, mas não um do outro
- Como Alice e Bob compartilham de uma mesma chave K , Alice poderia:
 - forjar uma mensagem;
 - criar um MAC usando K ;
 - dizer que essa mensagem veio do Bob.
- Bob poderia:
 - criar uma mensagem e MAC;
 - negar ter sido o autor.
- Em situações onde não há confiança plena entre emissor e receptor, precisamos de algo mais forte do que autenticação: **assinaturas digitais**.



Assinaturas Digitais

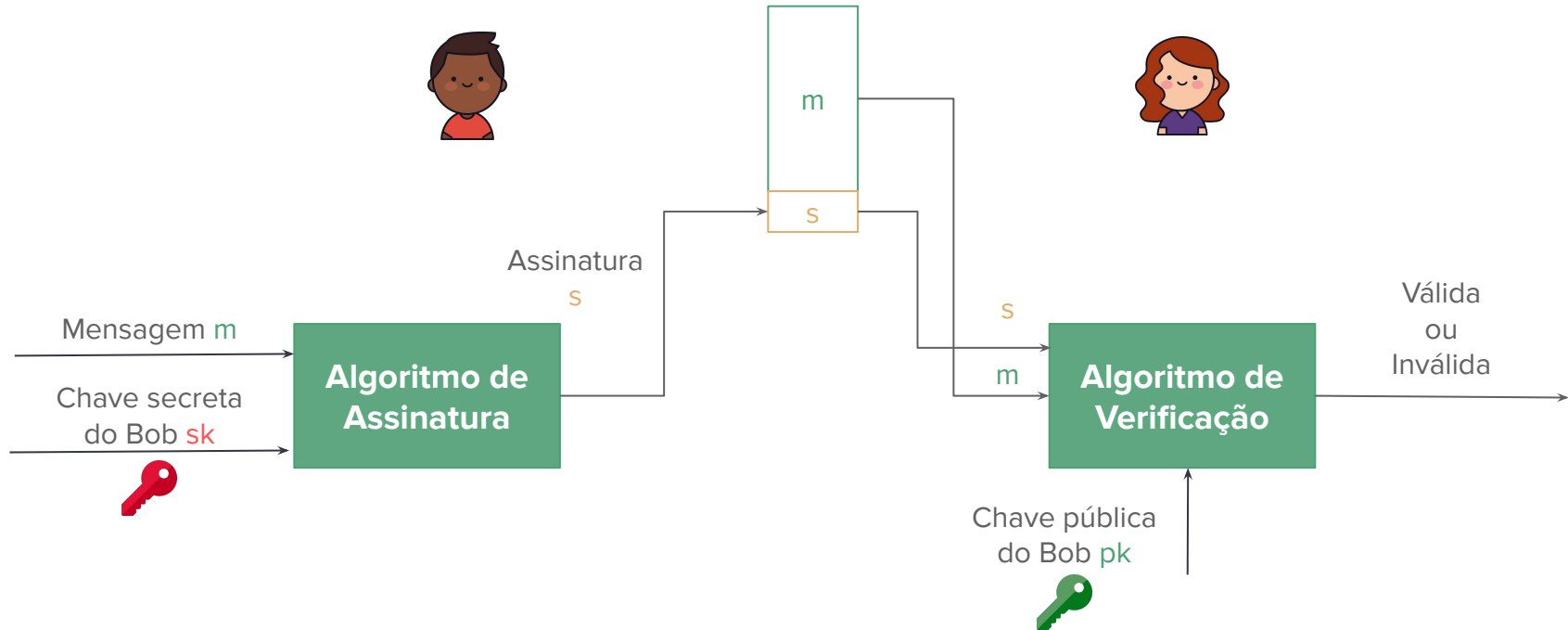
- Documentos digitais são facilmente manipuláveis
 - colar uma imagem de uma assinatura não provê segurança suficiente
 - qualquer pessoa pode colar essa imagem em diversos documentos
- Queremos obter o equivalente à assinaturas em papel
 - assinatura deve estar fortemente ligada ao documento
 - a identidade do assinante deve ser clara
 - apenas aquele assinante deve ser capaz de criar uma assinatura no nome dele
- Garantias com assinatura digital:
 - Integridade
 - Autenticidade
 - Não-repúdio

Características

Um esquema de assinatura possui os seguintes ingredientes:

- mensagem m a ser assinada
- par de chaves (uma pública (pk) e outra privada (sk))
- algoritmo de assinatura
- algoritmo de verificação de assinatura
- extra: uma função de hash

Assinatura Digital



Requisitos

- A assinatura deve depender de cada bit da mensagem
- Deve usar algo único do assinante
 - para prevenir falsificação e negação de assinatura por parte do assinante
- Deve ser fácil de produzir, reconhecer e verificar
- Deve ser computacionalmente inviável falsificar uma assinatura
 - tanto construir uma nova mensagem para uma assinatura existente
 - quanto construir uma assinatura falsificada para uma dada mensagem
- Dever ser possível reter uma cópia da assinatura de maneira prática

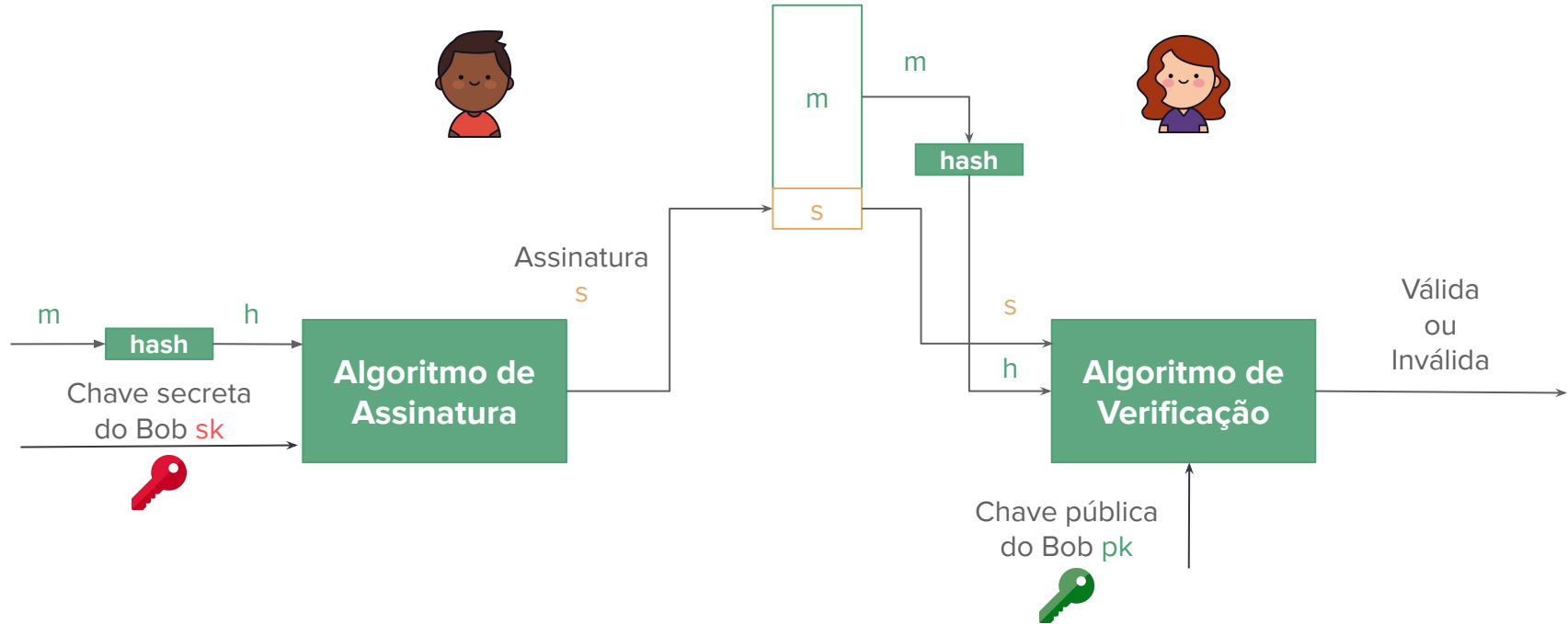
Algoritmos necessários

- Geração de chaves: $(pk, sk) = \text{KeyGen}$
- Assinatura: $s = \text{Sign}(m, sk)$
- Verificação: $\text{Verify}(m, s, pk) \begin{cases} \text{“válida” se } s = \text{Sign}(m, sk) \\ \text{“inválida” caso contrário} \end{cases}$

Assinatura digital e hash

- Normalmente, esquemas de assinatura são usados juntamente com uma **função de hash criptográfica**
- O hash da mensagem é assinado, ao invés da mensagem
 - Assinatura: $s = \text{Sign}(h(m), sk)$
 - Verificação: $\text{Verify}(h(m), s, pk)$

Assinatura digital e hash



Segurança em assinaturas digitais

- Dada uma mensagem m , deve ser computacionalmente inviável para qualquer outra pessoa além do Bob produzir uma assinatura válida s tal que $\text{Verify}(h(m), s, pk) = \text{válida}$
- Se um atacante conseguir calcular um par (m, s) válido para uma mensagem m que não foi previamente assinada por Bob, dizemos que isso é uma **falsificação**.
- Uma assinatura **falsificada** é uma assinatura válida produzida por alguém que não é dono da chave privada correspondente.

Modelos de ataques em assinaturas digitais

No caso de assinaturas digitais, os seguintes modelos de ataques são considerados:

- **key-only attack:** o atacante conhece apenas a chave pública
- **known message attack:** o atacante conhece uma lista de mensagens e assinaturas previamente assinadas por Bob $(m_1, s_1), (m_2, s_2), \dots$
- **chosen message attack:** o atacante escolhe uma lista de mensagens $m_1, m_2, \dots$ e consegue a assinatura do Bob nessas mensagens
- **selective forgery:** com probabilidade não-negligenciável, o atacante consegue produzir uma assinatura válida para uma mensagem m escolhida por alguém e não previamente assinada por Bob
- **existential forgery:** o atacante é capaz de criar uma assinatura válida para pelo menos uma mensagem, ou seja, criar um par (m, s) para uma m não previamente assinada por Bob.
- **total break:** o atacante é capaz de determinar a chave secreta do Bob

Segurança em assinaturas digitais

- Um esquema de assinatura não pode ser *incondicionalmente* seguro, já que o atacante pode testar todas as possíveis assinaturas s para dada mensagem m
 - ou seja, executar $\text{Verify}(h(m), s, pk)$ para todo possível s
- O objetivo é o de encontrar esquemas de assinatura que sejam *computacionalmente* seguros

Hash e segurança

- Precisamos garantir que o uso da função de hash não enfraqueça o esquema
- Por exemplo, dado o par (m, s) , um atacante pode tentar encontrar uma m' tal que $h(m) = h(m')$
 - nesse caso, a assinatura s seria uma assinatura válida para a nova mensagem m'
- Para evitar esse ataque, a função $h(.)$ precisa satisfazer as propriedades de hash já vistas
 - neste caso, precisa ser resistente à 2ª pré-imagem

Esquemas de assinatura digital

- O funcionamento interno dos algoritmos KeyGen, Sign e Verify dependem do esquema de assinatura utilizado
- Neste curso, estudaremos:
 - RSA
 - ECDSA
- Estes esquemas são apresentados no [FIPS 186-5](#)
- Existem diversos outros esquemas de assinatura digital
 - alguns, inclusive, resistentes contra computadores quânticos!

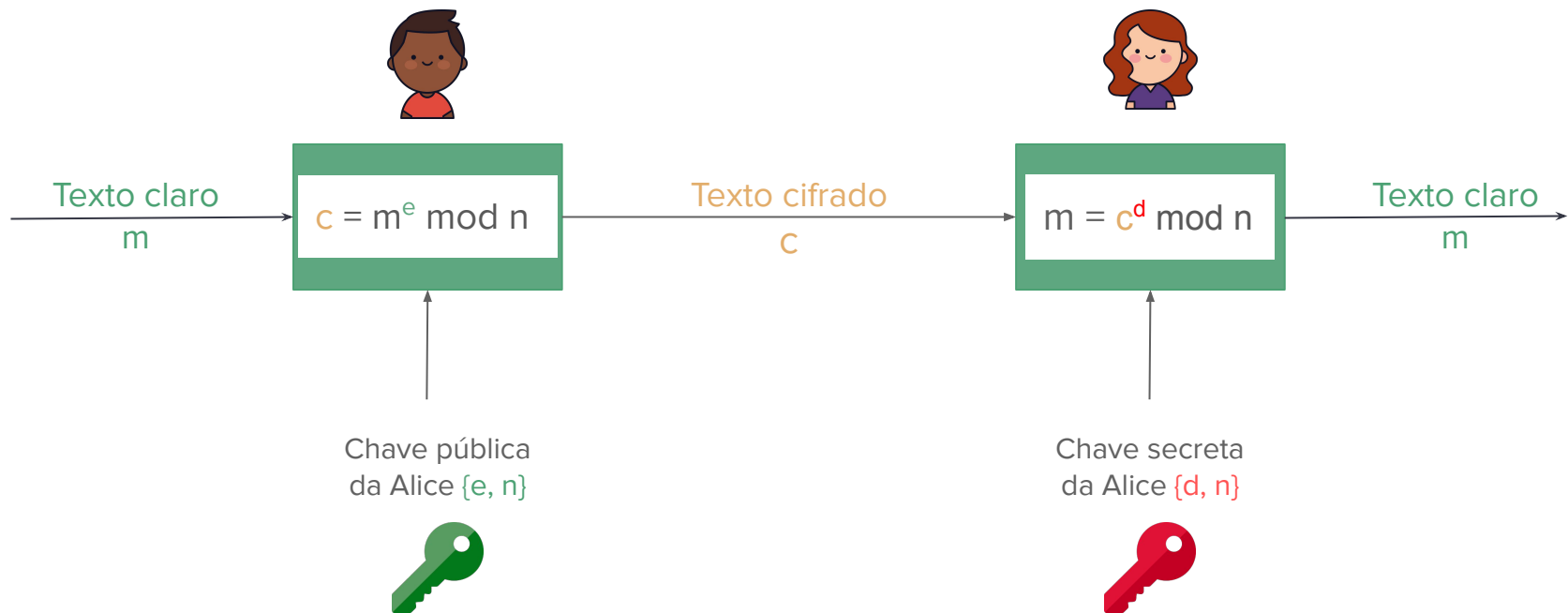
Sumário

- Assinaturas digitais
 - Princípios básicos
 - **Assinatura digital com RSA**
 - Assinatura digital com ECDSA
- Protocolos criptográficos
- Atividades práticas

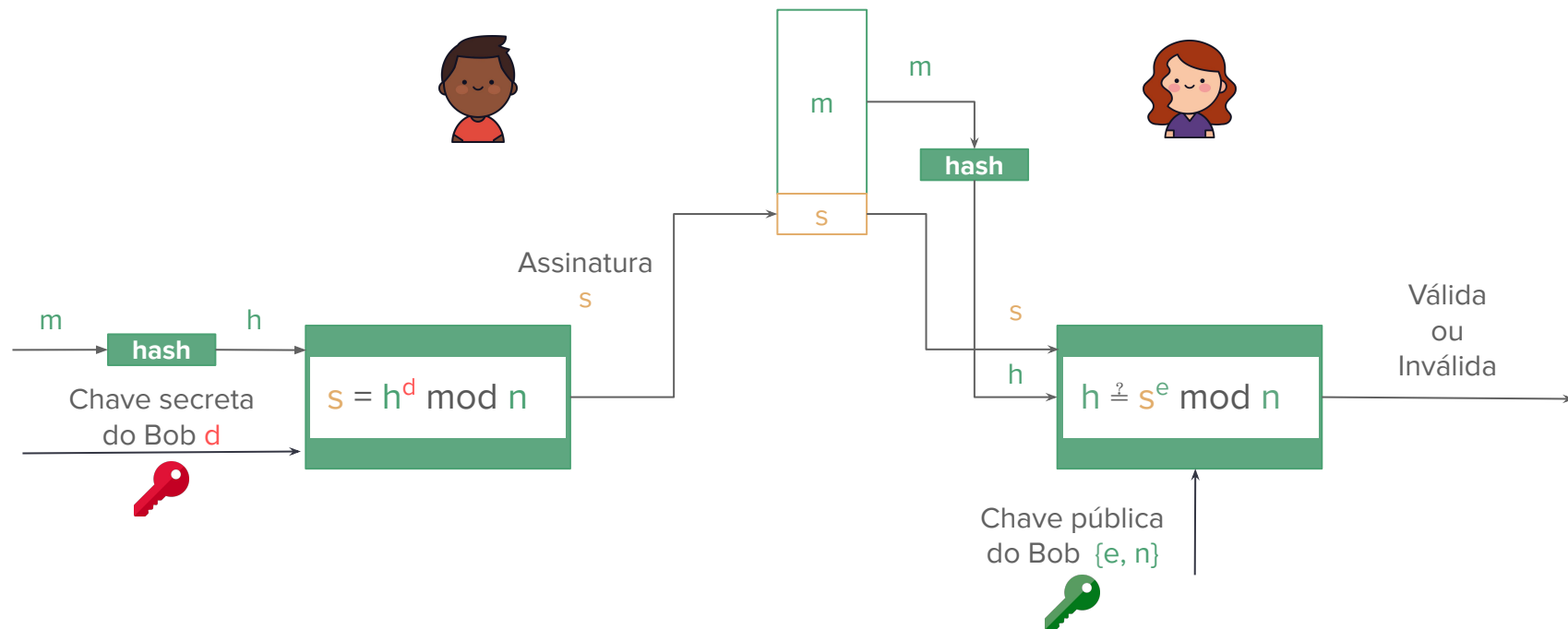
RSA

- Nas aulas anteriores, vimos geração de chaves, cifragem e decifragem com o criptossistema RSA
- Ele também pode ser utilizado para prover assinaturas digitais
 - com a modificação de que a chave privada é utilizada na geração da assinatura e a chave pública na verificação

RSA: Cifragem e Decifragem



RSA: Assinatura Digital



RSA: algoritmos

- Geração de chaves: KeyGen()
 - Sejam p e q números primos secretos, $n = pq$ e $\phi(n) = (p-1)(q-1)$, e e d tais que $ed \equiv 1 \pmod{\phi(n)}$
 - retorna $\{e, n\}$ como chave pública e $\{d, n\}$ como chave privada
- Assinatura: $s = \text{Sign}(h, d) = h^d \pmod{n}$
- Verificação: $\text{Verify}(m, s, e) \begin{cases} \text{“válida” se } h(m) \equiv s^e \pmod{n} \\ \text{“inválida” caso contrário} \end{cases}$

Considerações

- O expoente d precisa ser mantido em segredo
- O módulo n precisa ter pelo menos 2048 bits
- Apenas funções de hash aprovadas devem ser utilizadas na geração de assinaturas

Segurança do RSA

- O RSA que aprendemos aqui pode tem algumas vulnerabilidades
- A maioria dessas vulnerabilidades são evitadas se usarmos funções de hash seguras
- Outros cuidados também são tomados na implementação dos algoritmos
 - Assinaturas digitais com RSA são tão seguras quando o esquema de cifragem/decifragem RSA se acompanhadas de uma formatação com *padding* e aleatoriedade
 - Esse esquema é conhecido como RSA-PSS (RSA Probabilistic Signature Scheme)
 - Mais detalhes: [FIPS 186-5](#)

Segurança do RSA

Exemplo 1: um atacante pode construir uma assinatura válida para uma mensagem se ele primeiro escolher s e depois calcular a mensagem como $m = s^e \pmod{n}$.

- neste caso, a mensagem m possivelmente não teria um significado relevante
- além disso, o uso de funções de hash evitaria esse problema, pois $h = s^e \pmod{n}$, e o atacante precisaria descobrir m tal que $h = h(m)$
- note que, aparentemente, não há uma forma de escolher o m e depois criar a assinatura falsificada s , caso contrário o RSA seria inseguro.

Segurança do RSA

Exemplo 2: Um atacante obtém um par válido (m,s) previamente assinado por Bob.

- o atacante calcula $h = h(m)$ e tenta encontrar $m' \neq m$ tal que $h(m') = h(m)$
- a assinatura s também será válida para m'
- esse seria um *existential forgery* usando um *known message attack*
- o ataque pode ser prevenido usando uma função de hash resistente à segunda pré-imagem

Segurança do RSA

Exemplo 3: Um atacante pode encontrar duas mensagens $m \neq m'$ tal que $h(m) = h(m')$ e persuadir Bob a assinar m .

- essa assinatura gerada por Bob seria também uma assinatura válida para m'
- esse seria um *existential forgery* usando um *chosen message attack*
- o ataque pode ser prevenido usando uma função de hash resistente à colisões

Recomendações de segurança

- Quando uma função de hash e um algoritmo de assinatura são combinados, a segurança da assinatura é determinada pelo algoritmo mais fraco ([NIST SP 8000-57](#))
- Apenas funções de hash aprovadas devem ser utilizadas na geração de assinaturas
- Quando os parâmetros são gerados aleatoriamente (ex: e), deve ser usado um gerador de números aprovado
- O par de chaves gerado deve ser utilizado apenas para gerar e verificar assinaturas
- Chaves privadas devem ser protegidas de acesso não autorizado
- Chaves públicas devem ser protegidas de modificações não autorizadas
- Mais recomendações e detalhes em: [NIST FIPS 186-5](#)

Sumário

- Assinaturas digitais
 - Princípios básicos
 - Assinatura digital com RSA
 - **Assinatura digital com ECDSA**
- Protocolos criptográficos
- Atividades práticas

Elliptic Curve Digital Signature Algorithm (ECDSA)

- Esquema de assinatura digital baseado em curvas elípticas
- Variação do DSA para as curvas elípticas
- Mais recente que o RSA e DSA
 - proposto em 2009 pelo NIST na [FIPS 186](#)
- Vantagem de ter chaves muito menores

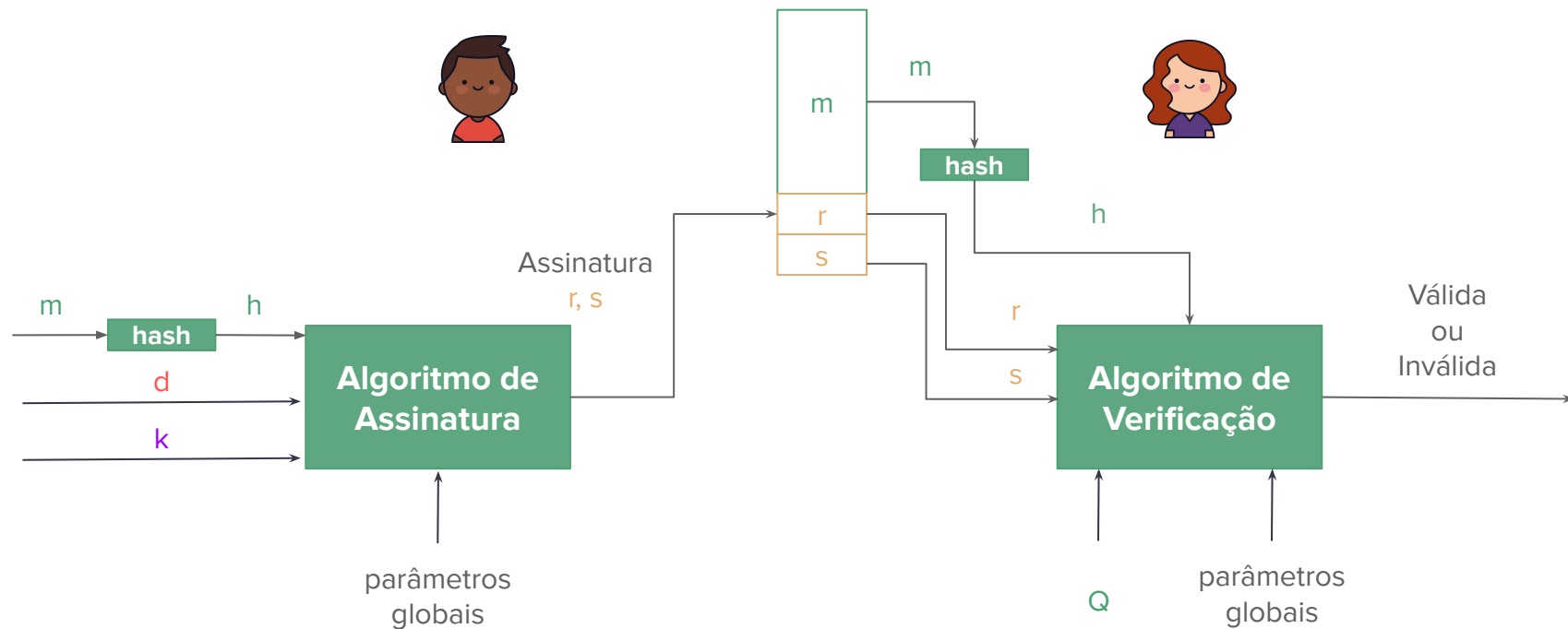
Relembrando...

- **Curva elíptica:** conjunto de pontos que satisfaz a equação $y^2 = x^3 + ax + b$
 - Chamamos esse conjunto de pontos de $E(a,b)$
 - Se nos restringirmos a valores em $\mathbb{Z}_p$, então chamamos de $E_p(a,b)$
- Sejam Q e P pontos em $E_p(a,b)$ e $k < p$
 - é fácil calcular Q dado k e P , $Q = kP$
 - é difícil determinar k dado Q e P
- Esse é o **problema do logaritmo discreto em curvas elípticas**
 - k é muito grande, tornando força-bruta inviável

ECDSA - overview

1. Todos os participantes do esquema precisam utilizar os mesmos parâmetros globais, que definem a curva a ser utilizada.
2. O assinante deve gerar um par de chaves. A chave privada é um número aleatório, enquanto a chave pública é um ponto da curva.
3. O hash da mensagem é assinado usando a chave privada e os parâmetros públicos. A assinatura consiste em dois inteiros r e s .
4. Para verificar a assinatura, o verificador utiliza a chave pública, os parâmetros globais e o valor s . Ele gera um valor v que é então comparado com r .

ECDSA



ECDSA - KeyGen

- **Parâmetros globais**

- número primo q
- inteiros a e b que definem a curva $E_q(a,b)$ com equação $y^2 = x^3 + ax + b$
- ponto $G = (x_g, y_g)$ em $E_q(a,b)$ cuja ordem é um número grande n
 - ordem: menor inteiro n tal que $nG = O$

- **KeyGen**

- Chave privada: valor aleatório d , $0 < d < n$
- Chave pública: $Q = dG$, onde Q é um ponto em $E_q(a,b)$

ECDSA - Sign e Verify

- **Assinatura: $(r, s) = \text{Sign}(m, d)$**

- Selecione um inteiro aleatório k , $0 < k < n$
- $P = (x, y) = kG$
- $r = x \bmod n$.
 - se $r = 0$, volte ao primeiro passo.
- $s = [k^{-1}(H(m) + dr)] \bmod n$
 - se $s = 0$, volte ao primeiro passo.
- Retorne a assinatura (r, s) .

- **Verificação: $\text{Verify}(m, (r, s), a, b, q, n, G, Q)$**

- Verifica que r e s são inteiros entre 1 e $n-1$
- $w = s^{-1} \bmod n$
- $u_1 = H(m) w \bmod n$
- $u_2 = r w \bmod n$
- $X = (x_1, x_2) = u_1 G + u_2 Q$
 - se $X = O$, rejeite a assinatura
- $v = x_1 \bmod n$
- se $v = r$
 - retorne "válida"
- senão:
 - retorne "inválida"

Considerações finais

Corretude:

- Não é óbvio verificar que o valor calculado de v é igual ao valor recebido r
 - Ver detalhes em: W. Stallings. *Cryptography and network security*, capítulo 13.5.

Segurança:

- Graças à dificuldade de resolver o logaritmo discreto em curvas elípticas (ECDLP), é inviável recuperar k a partir de r e d a partir de s
 - ou seja, é inviável falsificar uma assinatura
- Um novo valor k precisa ser gerado a cada nova assinatura, que precisa ser mantido em segredo
- Os valores de k e k^{-1} podem ser calculados previamente, desde que mantidos em local seguro assim como a chave privada

Ver mais em: [FIPS 186-5](#)

Comparação

Table 10.3 Comparable Key Sizes in Terms of Computational Effort for Cryptanalysis (NIST SP-800-57)

Symmetric Key Algorithms	Diffie–Hellman, Digital Signature Algorithm	RSA (size of n in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15,360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key.

Imagem: W. Stallings. *Cryptography and network security*. Cap 10.4

[NIST SP 800-57](#)

Aplicações

- Utilizadas em ambientes com capacidades limitadas de comunicação
- Uso na prática
 - [TLS](#)
 - [Bitcoin](#)
 - [iMessage](#)

Sumário

- Assinaturas digitais
- **Protocolos criptográficos**
 - Conceitos básicos de protocolos criptográficos
 - Protocolos simétricos
 - Protocolos assimétricos
- Atividades práticas

Contextualização

- Os tópicos de criptografia estudados até o momento são fundamentais para a garantia de segurança de comunicações
- Mas eles sozinhos não resolvem todos os problemas
 - Precisamos de uma combinação de primitivas criptográficas para atingir os objetivos desejados
 - Confidencialidade, Integridade, Autenticação, Anonimato, Temporalidade, Não-repúdio
- Existe uma variedade muito grande de ataques que precisam ser levados em consideração
 - como escutas (*eavesdropping*), roubos de dados, alterações de dados (*tampering*) e personificação (*impersonation*).
 - Criptografia por si só não protege de todos esses ataques

Introdução aos Protocolos de Segurança

- Protocolos de segurança são fundamentais para estabelecer confiança em comunicações via redes de computadores
- Definem um conjunto de procedimentos para assegurar que os dados transmitidos sejam seguros
 - confidencialidade, integridade, autenticidade, etc.
- **Através deles, podemos:**
 - negociar algoritmos a serem utilizados;
 - autenticar os participantes da comunicação;
 - fazer um acordo seguro de chaves (de sessão, de cifragem, etc.);
 - estabelecer uma comunicação segura;
 - entre outros.
- **Exemplos:**
 - SSL/TLS, IPsec, SSH, Kerberos.

Autenticação mútua

- Autenticação mútua permite que os dois lados de um canal de comunicação verifiquem a identidade um do outro
 - Entre dois indivíduos, entre cliente e servidor, conexões entre dispositivos, etc.
 - Se preocupam com troca de chaves, confidencialidade e temporalidade
- Podem ser baseado em chave assimétrica, simétrica ou ambos

Sumário

- Assinaturas digitais
- Protocolos criptográficos
 - Conceitos básicos de protocolos criptográficos
 - **Protocolos simétricos**
 - Protocolos assimétricos
- Atividades práticas

Autenticação Mútua Simétrica

- Partes A e B desejam compartilhar uma chave simétrica entre si
- Consideramos que tanto A quando B compartilham uma chave simétrica com uma terceira parte confiável
- Passos do protocolo:
 - Acordo de chave simétrica
 - Autenticação de ambos os lados
- Exemplos:
 - Needham-Schroeder Shared Key
 - Denning-Sacco

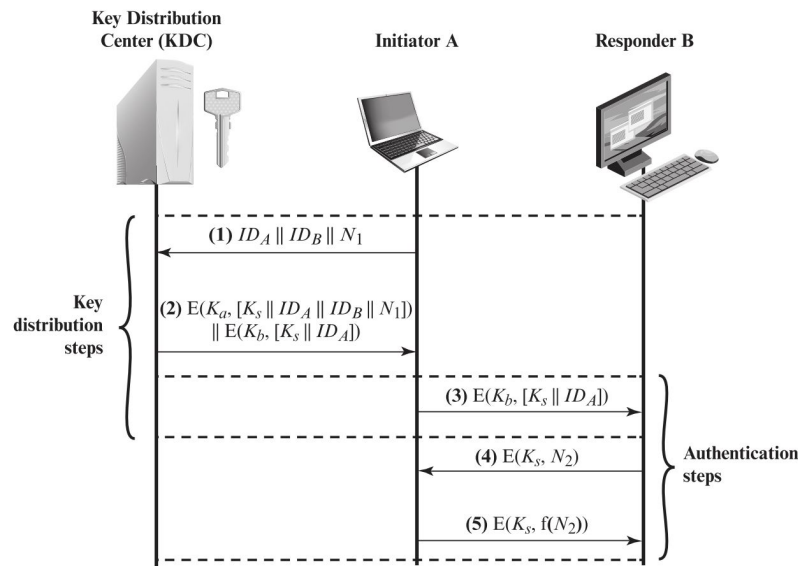
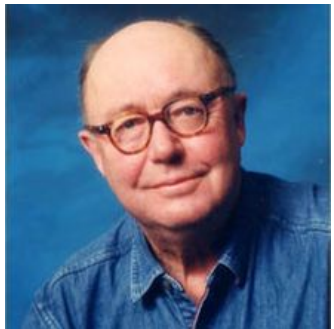


Figure 14.3 Key Distribution Scenario

Needham-Schroeder Shared Key (1978)

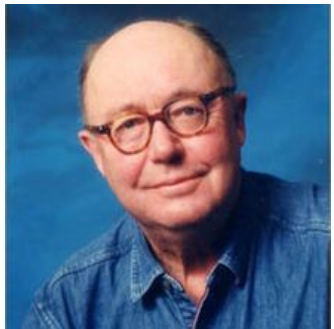


- **Objetivo:** distribuir chave de sessão K_{ab} entre partes A e B
- **Parâmetros:**
 - K_x : é a chave simétrica que X compartilha com o KDC
 - N_x : um nonce gerado por X
 - $E(k, m)$: cifragem de m com chave k
- **KDC:** Key Distribution Center
 - Assumimos que ele é confiável

1. $A \rightarrow \text{KDC}: A \parallel B \parallel N_a$
2. $\text{KDC} \rightarrow A: E(K_a, [K_{ab} \parallel B \parallel N_a \parallel E(K_b, [K_{ab} \parallel A])])$
3. $A \rightarrow B: E(K_b, [K_{ab}, A])$
4. $B \rightarrow A: E(K_{ab}, N_b)$
5. $A \rightarrow B: E(K_{ab}, N_b + 1)$

1. A pede ao KDC uma chave para falar com B
2. KDC retorna a chave K_{ab} e um ticket
3. A encaminha ao B o ticket gerado pelo KDC
4. Handshake: B testa se o A conhece a chave K_{ab}
5. Handshake: A responde ao teste

Needham-Schroeder Shared Key (1978)



- **Problemas:** Ataque de *replay*
 - Execução por tempo indeterminado
 - Não revogação de chaves de sessão
 - Se um atacante descobre uma chave antiga, ele pode re-enviar o passo 3 para B se passando por A
 - Se B não guardou todos os *nouces*/chaves usados com A, pode cair nesse ataque

1. $A \rightarrow \text{KDC}: A \parallel B \parallel N_a$
2. $\text{KDC} \rightarrow A: E(K_a, [K_{ab} \parallel B \parallel N_a \parallel E(K_b, [K_{ab} \parallel A])])$
3. $A \rightarrow B: E(K_b, [K_{ab}, A])$
4. $B \rightarrow A: E(K_{ab}, N_b)$
5. $A \rightarrow B: E(K_{ab}, N_b + 1)$

1. A pede ao KDC uma chave para falar com B
2. KDC retorna a chave K_{ab} e um ticket
3. A encaminha ao B o ticket gerado pelo KDC
4. Handshake: B testa se o A conhece a chave K_{ab}
5. Handshake: A responde ao teste

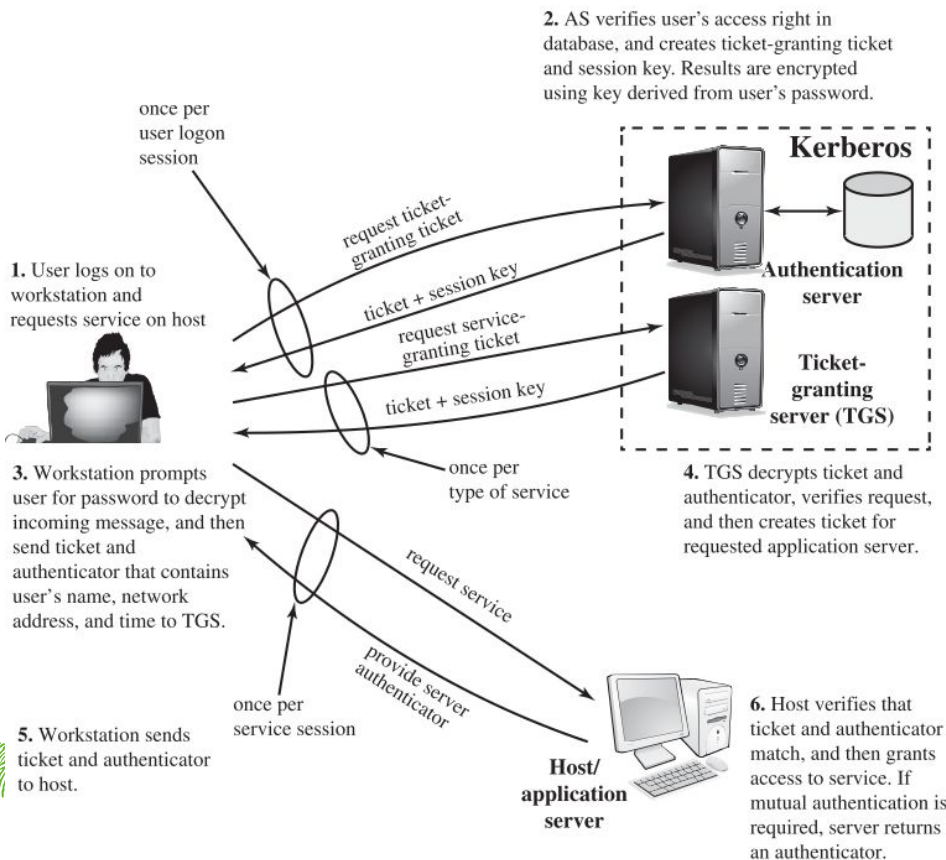
Denning-Sacco

- **Solução:** adição de um timestamp T
- Não temos o problema de executar por tempo indeterminado
 - Requer relógios sincronizados
 - Não sincronia pode gerar ataques de replay
- Outros protocolos surgiram para resolver esses problemas

1. $A \rightarrow KDC: A \parallel B$
2. $KDC \rightarrow A: E(K_a, [K_{ab} \parallel B \parallel T \parallel E(K_b, [K_{ab} \parallel A \parallel T])])$
3. $A \rightarrow B: E(K_b, [K_{ab} \parallel A \parallel T])$
4. $B \rightarrow A: E(K_{ab}, N_b)$
5. $A \rightarrow B: E(K_{ab}, N_b + 1)$

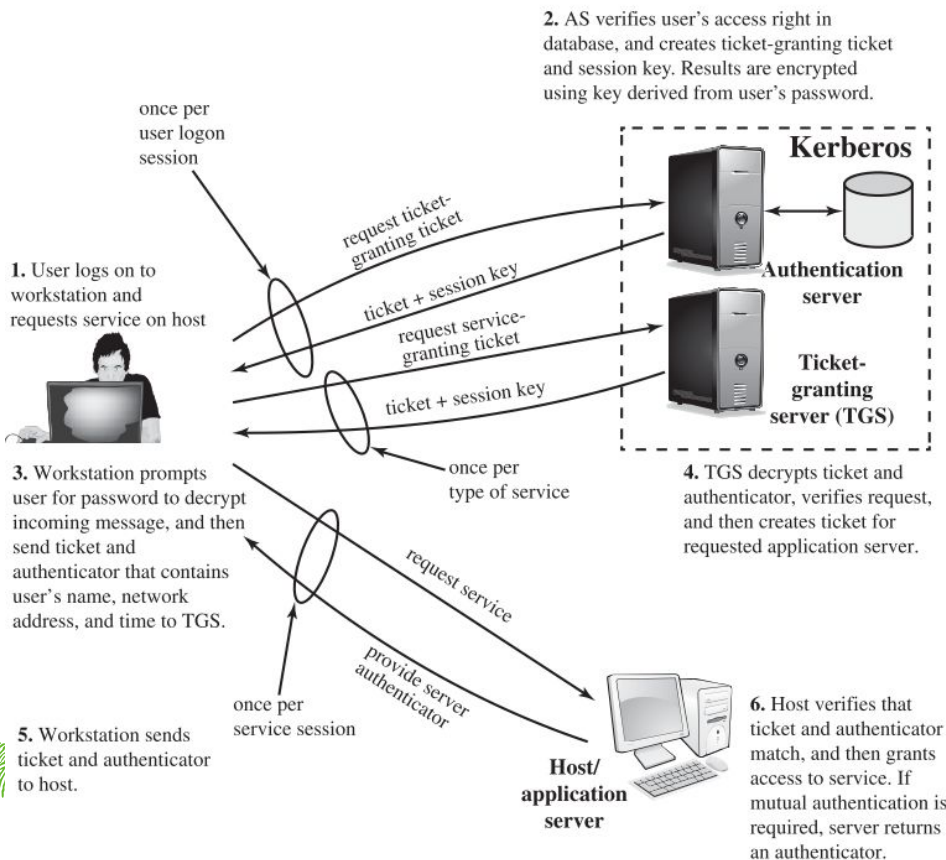


Kerberos



- Serviço de autenticação desenvolvido pelo MIT na década de 1980 é o protocolo mais usado do planeta
- Baseado na idéia de *tickets*
- Usa um modelo cliente-servidor
 - Servidor de autenticação (AS)
 - Servidor de tickets (TGS)
- Provê autenticação mútua
- Proteção contra vazamentos e ataques de repetição
 - Cifragem e timestamp
- Hoje em dia, sua maior aplicação é em ambientes corporativos.
- Usado no Apache, Oracle Database, MAC OS e iOS, Microsoft Active Directory

Kerberos



- **Aplicação:** ambiente distribuído no qual usuários e workstations desejam acessar através da rede serviços em servidores distribuídos
- **Possibilidades de ameaças:**
 - Atacante consegue acesso a uma workstation e finge ser um outro usuário
 - Atacante muda o endereço de rede de uma workstation para impersonar outra workstation
 - Atacante ouve as trocas de mensagens de outros usuários para fazer ataque de replay para conseguir acessar serviços ou atrapalhar operações

Sumário

- Assinaturas digitais
- Protocolos criptográficos
 - Conceitos básicos de protocolos criptográficos
 - Protocolos simétricos
 - **Protocolos assimétricos**
- Atividades práticas

Autenticação Mútua Assimétrica

- Criptografia de chave pública é ineficiente, portanto, pouco utilizada para cifragem direta de dados grandes
- Um dos mais importantes usos de criptografia assimétrica é o de cifrar chaves simétricas
- Consideramos que ambas as partes possuem um par de chaves, uma privada e outra pública
- Exemplos
 - NS Public Key
 - SSL/TLS

Protocolo de Merkle

- **Objetivo:** distribuir chave de sessão K_s entre partes A e B
- Apenas A possui um par de chaves (PU_a e PR_a)
- **Benefícios:**
 - Esquema extremamente simples que garante o compartilhamento seguro de K_s
 - A e B podem se comunicar usando K_s e depois descartam a chave
- **Problemas:**
 - Não garante autenticação mútua
 - Ataques de man-in-the-middle
 - atacante intercepta a comunicação em (1) e obtém K_s sem que A e B saibam

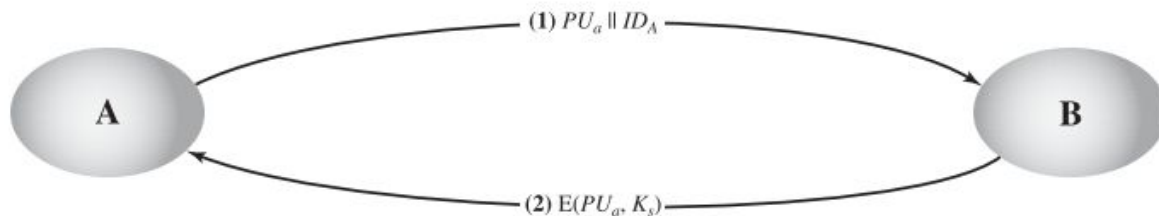
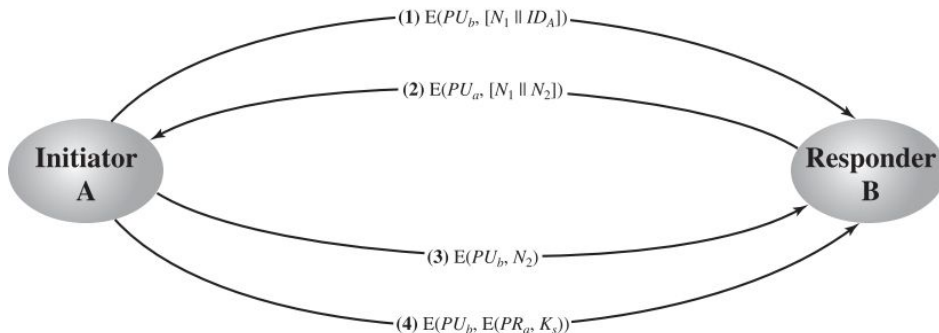


Figure 14.7 Simple Use of Public-Key Encryption to Establish a Session Key

Needham-Schroeder Public Key

- **Objetivo:** distribuir chave de sessão K_s entre partes A e B
- Tanto A quanto B possuem respectivas chaves públicas PU e privadas PR
- Garante autenticação mútua e confidencialidade no compartilhamento de uma chave de sessão
 - Passos (1), (2) e (3) garantem autenticação mútua, passo (4) compartilha a chave de sessão
- **Problemas:**
 - Execução por tempo indeterminado
 - Como ter certeza que a chave pública do A é de fato PU_a ?
- Ver outros protocolos no livro (Capítulo 15.4)



Transport Layer Security (TLS)

- **História:**
 - Sucessor do Secure Sockets Layer (SSL)
 - Desenvolvido inicialmente pela Netscape em 1995
 - Versões 1.0, 1.1, 1.2 e 1.3
- **O que é:**
 - Serviço de propósito geral implementado como um conjunto de protocolos
 - Garante confidencialidade, integridade e autenticidade das comunicações
 - Fica entre a camada de transporte (TCP) e a camada de serviços
- **Importância:**
 - HTTPS, VPNs, e-mail seguro, etc.
 - Essencial para transações online e proteção de dados



Imagem: [Cloudflare](#)

Resumindo o TLS

- Tem por objetivo de garantir a privacidade, autenticação e integridade de dados nas comunicações.
- A identidade das partes é **autenticada** usando criptografia de chave pública. Pode ser mútua ou pelo menos pelo lado do servidor
- **Conexão privada** ao compartilhar uma chave de sessão simétrica e utilizá-la para criptografar os dados transmitidos
- É possível verificar a **integridade** de cada mensagem transmitida.
- Com todas essas etapas, garantimos uma conexão segura entre cliente e servidor.

TLS - Overview

- Cliente e servidor se apresentam e decidem os algoritmos criptográficos que serão utilizados
- Servidor se autentica para o cliente com seu certificado digital
- Cliente verifica a validade do certificado
- Cliente e Servidor começam o processo de acordo de chaves
- Continuam a comunicação de forma cifrada usando a chave escolhida/calculada

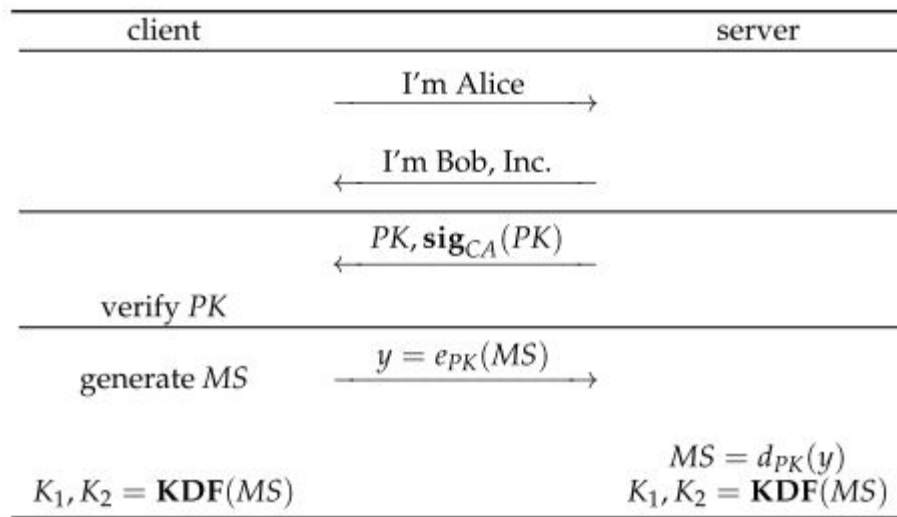


FIGURE 12.1: Setting up a TLS Session

Imagem: D. Stinson e M. Paterson. *Cryptography: Theory and Practice*. 4a edição. Capítulo 12.1.

Conjunto de protocolos do TLS

- Record Protocol
- Change cipher spec
- Alert protocol
- Handshake protocol

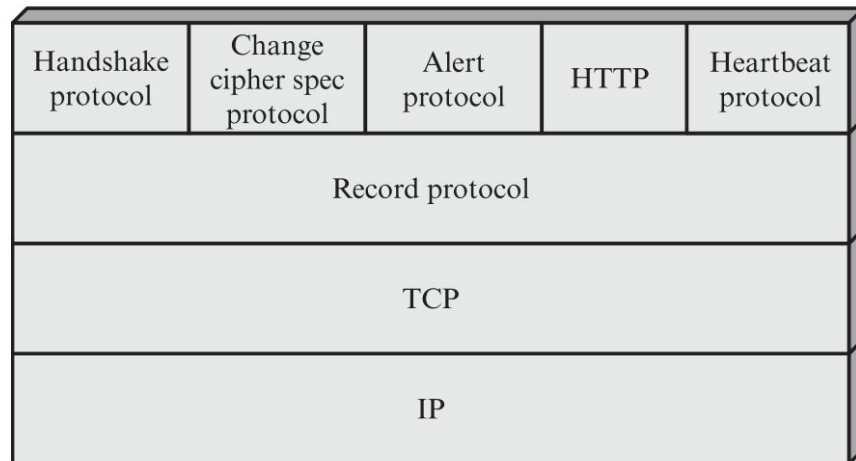


Figure 17.2 TLS Protocol Stack

Imagem: W. Stallings. *Cryptography and network security*. Cap 17.2

Conjunto de protocolos do TLS

Record Protocol: provê confidencialidade e integridade para conexões TLS

- confidencialidade através da cifragem
- integridade utilizando MACs
- Fragmenta a mensagem em blocos, protege os blocos e transmite o resultado
- Verifica, decifra e combina os dados recebidos, depois encaminha para o usuário

Change cipher spec: uma mensagem de 1 byte que sinaliza que o conjunto de cifras a ser usado nesta conexão deve ser atualizado.

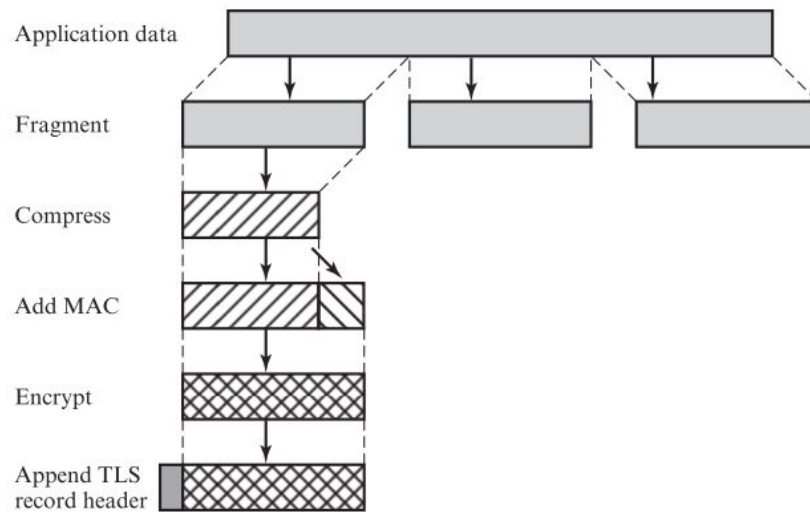


Figure 17.3 TLS Record Protocol Operation

Imagem: W. Stallings. *Cryptography and network security*. Cap 17.2

Conjunto de protocolos do TLS

Alert protocol: Usado para notificar erros ou eventos importantes durante a sessão TLS.

- alertas são comprimidos e cifrados de acordo com o estado atual da comunicação
- **níveis:** *warning* e *fatal*
- **classes:**
 - *closure alerts*: indicam que a conexão será finalizada/cancelada
 - *error alerts*: enviado pela parte que detectou o erro. Ambas as partes devem imediatamente finalizar a conexão
- Ver mais detalhes em: [RFC8446](https://tools.ietf.org/html/rfc8446)

```
enum { warning(1), fatal(2), (255) } AlertLevel;
```

```
enum {  
    close_notify(0),  
    unexpected_message(10),  
    bad_record_mac(20),  
    record_overflow(22),  
    handshake_failure(40),  
    bad_certificate(42),  
    unsupported_certificate(43),  
    certificate_revoked(44),  
    certificate_expired(45),  
    certificate_unknown(46),  
    illegal_parameter(47),  
    unknown_ca(48),  
    access_denied(49),  
    decode_error(50),  
    decrypt_error(51),  
    protocol_version(70),  
    insufficient_security(71),  
    internal_error(80),  
    inappropriate_fallback(86),  
    user_cancelled(90),  
    missing_extension(109),  
    unsupported_extension(110),  
    unrecognized_name(112),  
    bad_certificate_status_response(113),  
    unknown_psk_identity(115),  
    certificate_required(116),  
    no_application_protocol(120),  
    (255)  
}  
} AlertDescription;  
  
struct {  
    AlertLevel level;  
    AlertDescription description;  
} Alert;
```

Conjunto de protocolos do TLS

Handshake protocol: negociação dos parâmetros de segurança de uma conexão

- Utilizado antes de qualquer transmissão de dados
- Consiste em uma série de mensagens trocadas entre cliente e servidor
- Cliente e servidor se autenticam um para o outro e definem os algoritmos criptográficos que serão usados
- Cada mensagem trocada tem os campos como tipo, tamanho e conteúdo

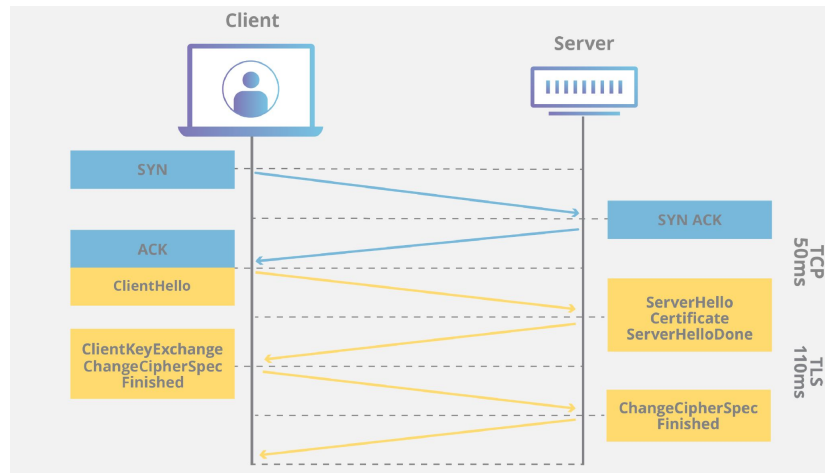


Imagem: [Cloudflare](#)

Handshake protocol

- Client hello:
 - versão do TLS compatível
 - conjuntos de algoritmos criptográficos compatíveis
 - grupos suportados para EC
 - algoritmos de assinatura
 - string de bytes aleatórios
- Server hello:
 - certificado SSL do servidor
 - algoritmos criptográficos escolhidos
 - string de bytes aleatórios
- Autenticação
 - cliente verifica o certificado SSL do servidor com a AC que o emitiu
 - cliente garante a interação com o verdadeiro proprietário do domínio

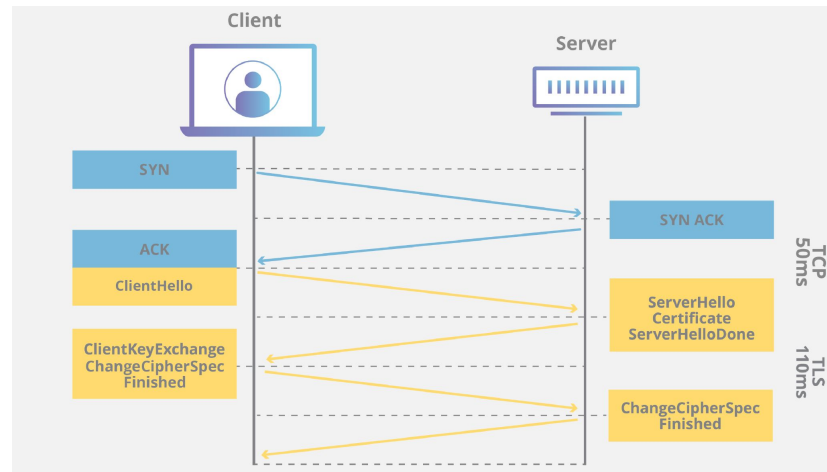


Imagem: [Cloudflare](https://cloudflare.com)

Handshake protocol

- Acordo de chaves
 - cliente envia uma *pré-master secret* cifrada com a chave pública do servidor
 - a *pré-master secret* é utilizada pelo cliente e servidor para gerarem a mesma chave de sessão
- Comunicação é cifrada a partir daqui

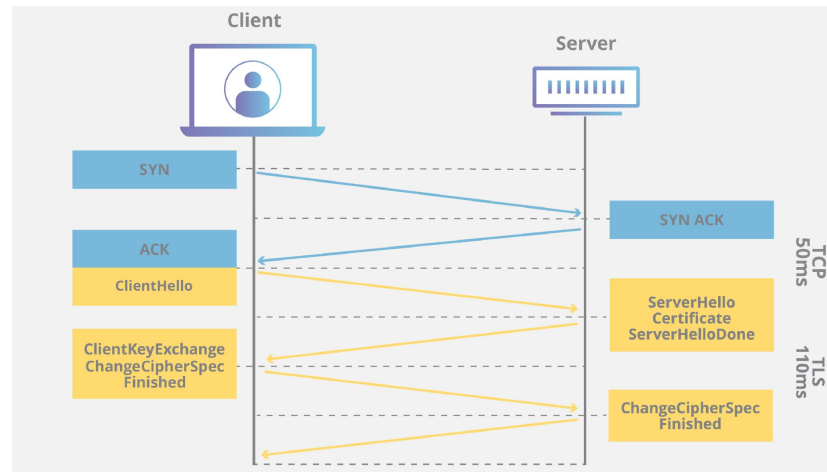


Imagem: [Cloudflare](https://cloudflare.com)

Handshake protocol

- Alternativa: ephemeral Diffie-Hellman
 - servidor envia o seu certificado e junto dele a assinatura de todas as mensagens até aqui
 - cliente verifica a assinatura e certificado
 - cliente envia parâmetros DH para para o servidor
 - cliente e servidor calculam a chave de sessão usando DH

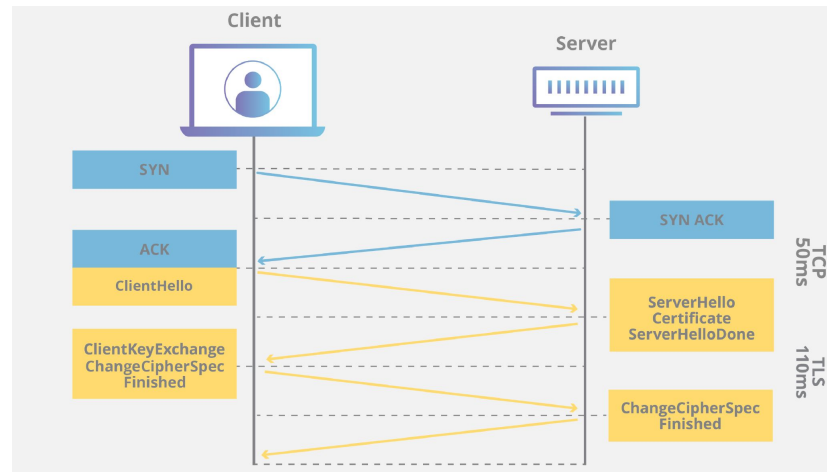


Imagem: [Cloudflare](#)

Modo 0-RTT

- Ocorre caso cliente e servidor já tenham se conectado antes
- derivam outra chave secreta a partir das informações da primeira sessão
- não há necessidade de comunicação para handshake

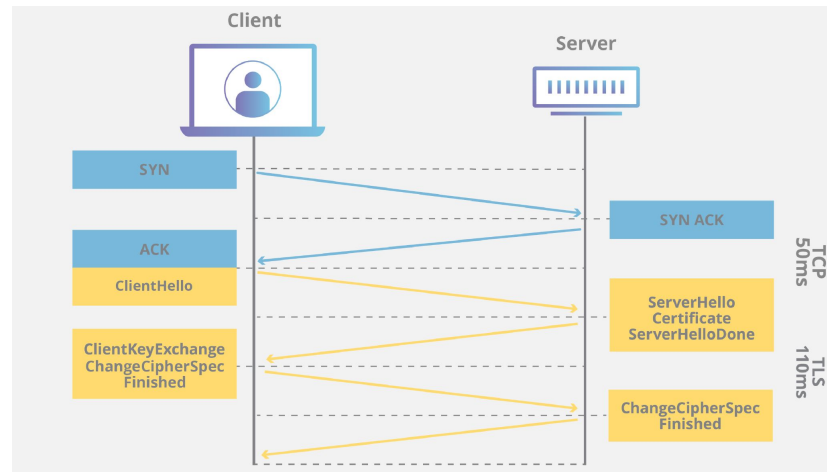


Imagem: [Cloudflare](#)

Cipher suites

- Combinações de algoritmos de criptografia utilizados pelo TLS para garantia de uma conexão segura
- Cifragem:
 - TLS_AES_128_GCM_SHA256
 - TLS_AES_256_GCM_SHA384
 - TLS_CHACHA20_POLY1305_SHA256
- Assinatura digital
 - rsa_pkcs1_sha256
 - rsa_pss_rsae_sha256
 - ecdsa_secp256r1_sha256



Imagem: [Cloudflare](#)

HTTPS

- HTTPS (HTTP over SSL) se refere à combinação de SSL com HTTP para a implementação de uma comunicação segura entre browser e servidor web.
- HTTP utiliza a porta 80 enquanto o HTTPS utiliza a 443
- Ao utilizar HTTPS, as seguintes informações são cifradas:
 - URL
 - conteúdo da comunicação entre cliente e servidor
 - dados preenchidos pelo usuário
 - cookies
 - cabeçalhos HTTP
- Mais informações: [RFC2818](#)



Imagem: [Cloudflare](#)

IPSec

- Criado na década de 90 pelo *Internet Engineering Task Force*
- IPSec é uma extensão do protocolo IP
- O protocolo IP é o principal protocolo de roteamento de pacotes na internet
 - se preocupa com conexão e entrega de pacotes
 - não utiliza criptografia por *default*
- IPSec adiciona criptografia ao protocolo IP
 - garantia de integridade, autenticidade e confidencialidade dos pacotes
- Normalmente utiliza a porta 500
- Mais detalhes: [RFC6071](#)

Como funciona o IPSec?

Visão geral:

- Emissor inicia uma transmissão para o destinatário utilizando IPSec
- Emissor e destinatário negociam os requisitos para o estabelecimento de uma conexão segura
 - Algoritmos, chaves, e outros parâmetros do protocolo SA
- Pacotes são enviados e recebidos de forma criptografada
 - decifragem, verificação de integridade e autenticidade são feitas
- Conexão IPSec é encerrada

SSH

- O *Secure Shell* (SSH) é um protocolo que permite o acesso e gerenciamento de dispositivos remotos de maneira segura
- Foi criado em 1995 Tatu Ylönen para resolver problemas de segurança em uma universidade finlandesa
- Normalmente baseado em uma arquitetura cliente/servidor
- Normalmente utiliza a porta 22

Como funciona o SSH?

Visão geral:

- Um cliente inicia uma conexão SSH em um servidor
- Cliente e servidor negociam algoritmos e estabelecem uma chave de sessão
- Uma autenticação do servidor é feita
- Cliente envia suas credenciais (login/senha) para autenticação no servidor
- A partir daqui, a conexão SSH é estabelecida e está pronta para uso

Comparação

- Tanto o TLS quando o IPSec são utilizados para proteger dados trafegados entre dispositivos e proteger tráfego de rede
- A diferença entre eles é a camada no qual operam
 - TLS fica entre a camada de transporte (TPC) e a de serviços
 - IPSec opera na camada de rede
- Já o SSH é um protocolo utilizado para administrar dispositivos remotos de forma segura

Sumário

- Assinaturas digitais
- Protocolos criptográficos
- **Atividades práticas**

Atividade: assinatura RSA com openssl

- Vamos utilizar as chaves RSA criadas anteriormente:
 - `openssl genrsa -aes256 -out seunome.privada.pem 2048`
 - `openssl rsa -pubout -in seunome.privada.pem -out seunome.publica.pem`
- Crie um arquivo de texto qualquer com uma mensagem
 - `echo "Mensagem autentica" > msg.txt`
- Assine a mensagem usando a chave privada
 - `openssl dgst -sha256 -sign seunome.privada.pem -out assinatura.bin msg.txt`
- Verifique a assinatura
 - `openssl dgst -sha256 -verify seunome.publica.pem -signature assinatura.bin msg.txt`

Atividade: assinatura RSA com openssl

- Modifique o arquivo de texto e tente verificar a assinatura novamente
 - o que acontece?
- Gere uma assinatura de um documento e peça para um colega verificar
 - que informações são necessárias?

Atividade: Gerando chaves com o openssl

- Os comandos abaixo já foram executados na aula de ECC!
 - caso tenha feito, utilize as chaves já geradas
- Verifique as curvas disponíveis no openssl:
`openssl ecparam -list_curves`
- Gere a chave privada com a curva escolhida:
`openssl ecparam -genkey -name <nome_da_curva> -out <nome_chave_privada>.pem`
- Extraia a chave pública:
`openssl ec -in <nome_chave_privada>.pem -pubout -out <nome_chave_pública>.pem`
- Visualize as chaves:
`openssl ec -in <nome_chave_privada>.pem -text -noout`
`openssl ec -in <nome_chave_pública>.pem -text -noout -pubin`

Atividade: assinando com ECDSA

- Crie um arquivo de texto qualquer com uma mensagem
 - `echo "Mensagem autentica" > msg.txt`
- Assine a mensagem usando a chave privada
 - `openssl dgst -sha256 -sign <nome_chave_privada>.pem -out assinatura.bin msg.txt`
- Verifique a assinatura
 - `openssl dgst -sha256 -verify <nome_chave_pública>.pem -signature assinatura.bin msg.txt`
- Modifique a mensagem e verifique a assinatura novamente

Referências

- W. Stallings. *Cryptography and network security*. 7a edição.
 - Definições, requisitos e ataques: 13.1
 - ECDSA: 13.5
 - RSA-PSS: 13.6
 - Protocolos de autenticação mútua: 15
 - TLS, HTTPS, SSH: 17.4
- D. Stinson e M. Paterson. *Cryptography: Theory and Practice*. 4a edição.
 - Introdução: 1.2.2
 - RSA: 8.1, 8.2
 - ECDSA: 8.4.3
 - TLS: 12.1.1
- Joachim von zur Gathen. *CryptoSchool*. 1a edição.
 - Assinaturas: 8
- Recomendações para gerenciamento de chaves: [NIST SP 800-57](#)
- [RFC6071](#): IP Security (IPsec) and Internet Key Exchange (IKE) Document Roadmap
- [RFC2409](#): The Internet Key Exchange (IKE)
- [RFC4302](#): IP Authentication Header
- [NISTIR 7966](#): Security of Interactive and Automated Access Management Using Secure Shell (SSH)
- [RFC4253](#): The Secure Shell (SSH) Transport Layer Protocol
- imagem: Flaticon.com